

How to find Classes and Objects?

1

1



每年都差不多，供應商的例子大概只有 1/3 的人分析是答對的

2

2



3

3

THINGS

- An object is some **Thing** in the world of the user
 - Car, Train, Elephant, Sale, Invoice, Division, Account, etc.
 - Physical or Conceptual
- It has **Attributes** that describe it:
 - Make, Model, Year,
 - Color, Weight,
 - Serial Number, License Number
- It has **Relationships** to other objects:
 - e.g., to a Person object, i.e., the Owner.
- It has **Behavior** that it can do:
 - Create
 - Change Color
 - Change Owner
 - Destroy Itself (Suicide?)

Entity

Object

4

9.1. The Importance of This Step.

Jacobson's Three Types

- **Entity Objects** - things in the users' real world
 - *Concrete* objects - Employee, Product, Tool, etc.
 - *Conceptual* objects - Corporation, strategy, membership, approval, etc.
 - *Event and State* objects - Purchase, delivery, arrival, ownership, status, etc.
- **Interface Objects** - To encapsulate the details of communication protocols, and the like. GUIs,
- **Control Objects** - To carry complex methods that don't have a class they obviously belong in.

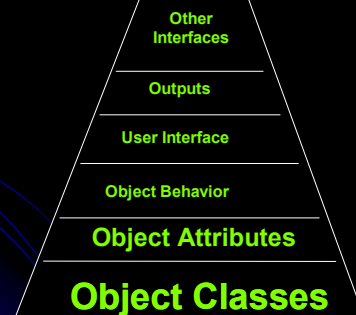
5

5

① 5.1. Real-World vs Data-World Objects

Jacobson's Three Types

Interface Objects, Advantages:



6

The KRB Seven-Step Method

Step 1 Candidate Classes

- Several ways to find *nouns* that might turn out to be names of classes that matter to your project.

Step 2 Define Classes

- Here we check for the meaning the *users* attach to each noun, and decide whether it is important *for this project*.

Step 3 Establish Associations

- Checking how these things interact gives us a picture of the pathways that exist throughout the data

Step 4 Expand Many-to-Many Associations

- An essential step that generates much discussion and *more classes*.

7

7

9.2. The KRB Seven-Step Method Overview

Step 5 Attributes

- We list out all the data fields the users need to keep *about* each *thing* (i.e., *about* each type of entity).

Step 6 Normalization

- A technical consideration, with a different relevance in the object paradigm. This step is done behind closed doors without the users.

Step 7 Operations (i.e., Behavior)

- This is the additional step that distinguishes objects from entities and earlier models. This part is the subject of Chapters 10 and 11.

8

8

9.2. The KRB Seven-Step Method, Step 1 Candidate Classes

Definition:

Candidate Classes:

Nouns that are possible class names, being considered for inclusion in the project.

Just find all the names!

9

9

The sources of entity objects

1. Client Interviews
2. Nouns from the Documentation
3. Brainstorming

10

10

The sources of interface objects

User Interfaces:

- Your classes will be defined by your GUI tool.

Data Communications Interfaces:

- If you buy a communications class library, your classes are already defined for you.
- The published comm protocol should be filed with or referred from your project documentation.

Real-world systems to control or monitor:

- Industrial Processes, etc.
- Define a single-instance class to represent the external system.

11

11

Control classes

- Will mostly be discovered in the Design Phase of the OODLC.
- As with anything else, if the topic arises earlier during the Analysis Phase,
- Use your judgement;
- Either do it now, or
- Make serious notes to *ensure* it gets done later. (Don't ever miss it!)

12

12

9.2. The KRB Seven-Step Method. Step ② Define Classes

Each Candidate Class must undergo three checks:

- A Real-World **Identifier**
("How do I tell one . . . from another?")
- A **Definition** ("What is a . . . ?")
- Sample **Attributes** and **Behaviors**
("What might I need to know *about* a . . . ?"
or "What can happen to a . . . ?")

13

13

9.2. The KRB Seven-Step Method. Step ② Define Classes

Question 1: A Real-World Identifier

- Objects have *Identity*,
- So they are distinct from each other,
- And there is usually some way to **tell them apart**.
- How do we tell one **Student** from another?
 - **Different names** - won't do, not unique.
 - Also, names and such are too sensitive to errors
 - Punctuation and case
 - Spelling and spacing
 - **Different faces** - scanning technology not yet good enough

14

14

9.2. The KRB Seven-Step Method. Step ② Define Classes

Question 1: A Real-World Identifier

Some Examples of Identifiers:

- | | |
|-----------------|----------------------------|
| • Student No. | • Building ID |
| • Customer No. | • Street Name |
| • Account No. | • City Name |
| • Licence No. | • State/Province ID |
| • Serial No. | • Vehicle ID No. (VIN) |
| • Product Code | • Inventory No. |
| • Category Code | • Room No. |
| • Vendor ID | • Legal Description (Land) |
| • Employee No. | • Policy No. |

15

15

9.2. The KRB Seven-Step Method. Step ② Define Classes

Question 1: A Real-World Identifier

- So we try to find a unique identifier for our Candidate Class.
- If we can't, then it's definitely not a class.
- If we eventually do find one, then we move on to the next test,

Question 2: The Definition . . .

16

16

9.2. The KRB Seven-Step Method. Step ② Define Classes

Question 2: The Definition

- Employee: A person who works at XYZ Widgets.
 - Sounds reasonable, but do we simply delete the records when (s)he quits? Try this:
 - Employee: A person who works, or has worked, at XYZ Widgets.
 - Better, and this will work for most businesses.
 - But you may find a company that wants *prospective* employees to be kept in the database! Try this:
- Employee: A person who works, has worked, or may in the future work at XYZ Widgets.

17

17

9.2. The KRB Seven-Step Method. Step ③ Establish Associations

Here we ask the questions:

- "What does a Student *DO* to a Course?"
- *Enrolls in* it, of course.
- "What does a Customer *DO* to a Product?"
- A Customer *buys* a Product.
- "What does a Teacher *DO* to a Course?"
- A Teacher *teaches* a Course.

18

18

9.2. The KRB Seven-Step Method. Step ③ Establish Associations

Here we ask the questions:

- “What does a Student *DO* to a Course?”
- *Enrols in* it, of course.
- “What does a Customer *DO* to a Product?”
- A Customer *buys* a Product.
- “What does a Teacher *DO* to a Course?”
- A Teacher *teaches* a Course.

19

19

9.2. The KRB Seven-Step Method. Step ③ Establish Associations

- An *association* is an *inter*action between instances of two classes,
- Represented by a *verb*.
- The ones we want are those that describe *things that happen in the users' business*.
- We need a sentence that goes
“Object *verb* Object”
- *And makes sense* in the users' world.

20

20

9.2. The KRB Seven-Step Method. Step ③ Establish Associations

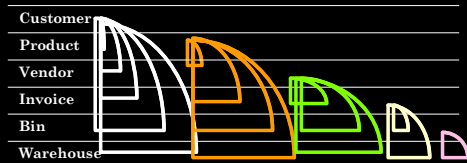
- There are two important parts to this sentence:

*The Verb, and
The Multiplicity.*

21

21

The KRB Seven-Step Method. Step ③ Establish Associations



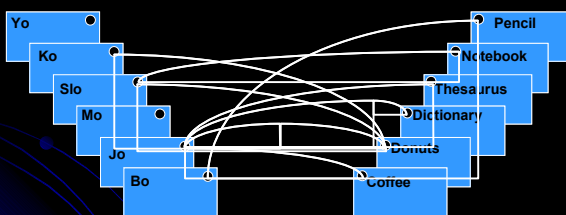
- Draw a line against the list of classes, each time you check out a possibility.
- This way you check out *each class* against *every class below it* in the list.

22

22

9.2. The KRB Seven-Step Method. Step ④ Expand Many-to-Many Associations

Take a look at this *card-and-string* database:

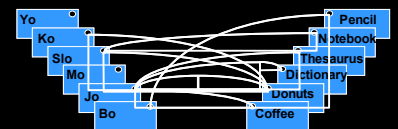


23

23

9.2. The KRB Seven-Step Method. Step ④ Expand Many-to-Many Associations

- This will allow us to do sales reports by either:
- Pulling a Customer card, and the strings will bring all the relevant Product cards,
- Or vice-versa.
- But there are problems . . .



24

24

9.2. The KRB Seven-Step Method.
Step ④ Expand Many-to-Many Associations

More realistically, we could perhaps write on the cards instead of using strings:

As you can easily see on Jo's card, we are likely to run out of room!

25

25

9.2. The KRB Seven-Step Method.
Step ④ Expand Many-to-Many Associations

And,

The data is recorded in *two* places, which is not good design, since it *will* get out of step (Murphy's Law)

26

26

9.2. The KRB Seven-Step Method.
Step ④ Expand Many-to-Many Associations

Not only that, but where would we write the date, quantity and selling price for each sale?

On the strings perhaps?

27

27

9.2. The KRB Seven-Step Method.
Step ④ Expand Many-to-Many Associations

We replace each string with a card and two strings:

28

28

9.2. The KRB Seven-Step Method.
Step ④ Expand Many-to-Many Associations

We replace each string with a card and two strings:

29

29

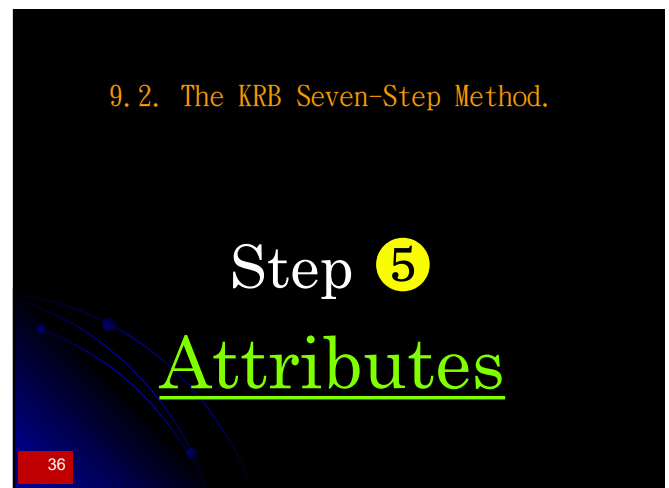
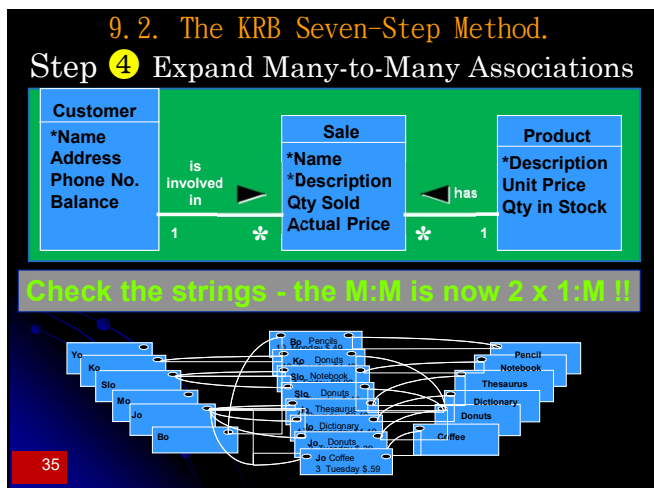
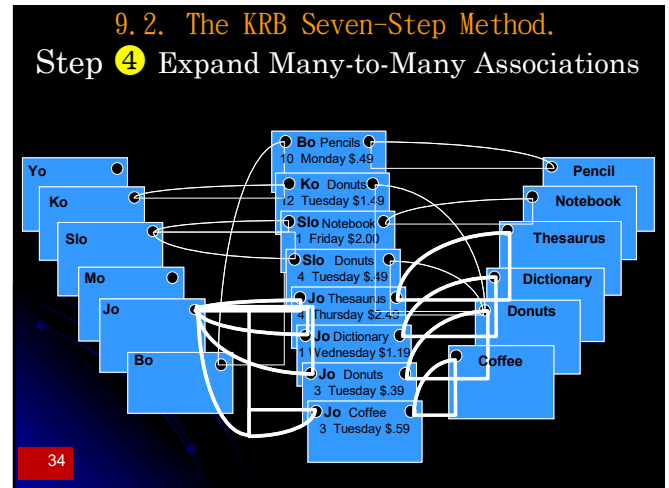
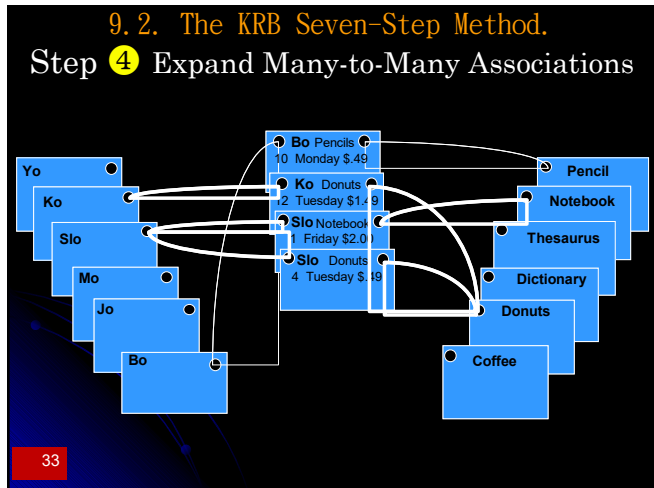
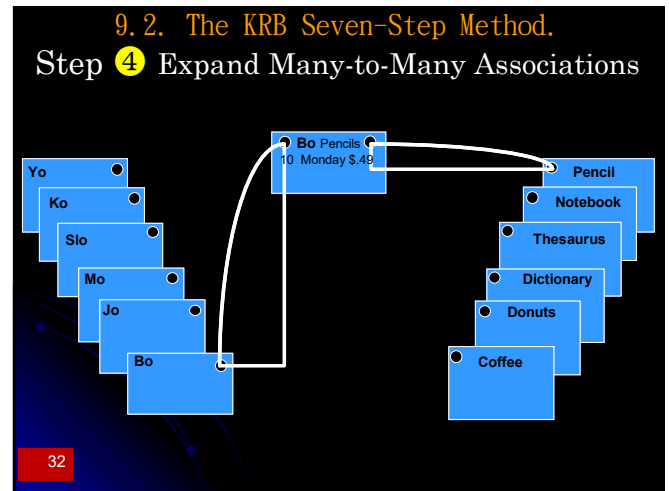
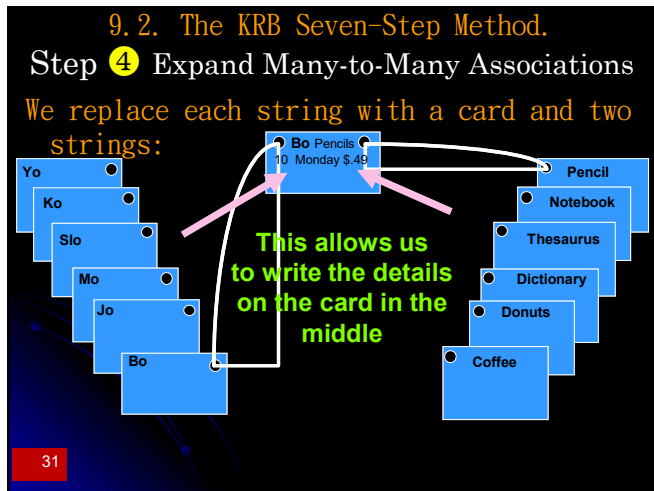
9.2. The KRB Seven-Step Method.
Step ④ Expand Many-to-Many Associations

We replace each string with a card and two strings:

This allows us to write the details on the card in the middle

30

30



9.2. The KRB Seven-Step Method.

Step 6

Normalization

37

37

9.2. The KRB Seven-Step Method.
Step 6 Normalization

Let's start with an example.

Say you need a sales report something like this:

Customer			Catalog		Unit Price	Date	Qty Sold	Actual Price	Extended Price
No.	Name	Address	No.	Description					
131	Jo Blo	13 May St	3A21	T-Shirt	12.49	03/01/98	45	10.00	450.00
179	Yo Yo	271 OK Ave	1B77	Sweats	15.00	01/03/98	12	15.00	180.00
212	Mu Mu	32 Saddle Rd	4X21	Pants	23.47	12/11/98	5	21.00	105.00
.	.	.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.	.	.

38

38

9.2. The KRB Seven-Step Method.
Step 6 Normalization

- What the *uninitiated* (read "*amateur*") database designer tends to do

is to build a relational table
that *mimics this report*.

- That is, it has the *same columns* as this report.
- In the object paradigm, the *equivalent error* would be to create a class with *attributes* that *match the columns* on the report.
- But what would we call this class?
- The best name would probably be something like "Sales" or "Sales Analysis."

But . . .

39

39

9.2. The KRB Seven-Step Method.
Step 6 Normalization

- The problem is that we have **three kinds of data in this report**.

- We have:

Data that describes a **Customer** (Cust No./Name/Address)Data that describes a **Product**

(Cat No./Description/Unit Price)

And data that describes a **Sale** (Date/Quantity/Actual and Extended Prices)

- Compare this situation with all the earlier models we have looked at,
- You'll see that **Customer**, **Product** and **Sale** should each be a *separate class* . . .

40

40

9.2. The KRB Seven-Step Method.
Step 6 Normalization*Like this!*

41

41

9.2. The KRB Seven-Step Method.

Step 7

Operations
(i.e., Behavior)

42

42

9.2. The KRB Seven-Step Method. Step ⑦ Operations (i.e., Behavior)

Six techniques for Finding Operations

1. By Inspection
2. Basic *CRUD*
3. Use Cases
4. Statechart Diagram
5. CRC Cards
6. *CRUD* Revisited

43

43

Step ⑧

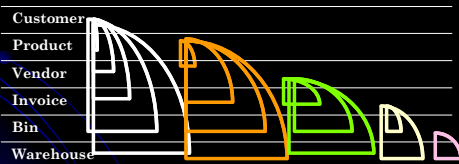
- Find the similarity (in behaviors) and construct inheritance hierarchy
- Make sure there are association linked to base classes (a.k.a polymorphism)

44

44

Case Study

- Draw the class diagram with methods
- Noun list and their definition
- Association check
- (Check polymorphism exists)



45

45

RUP

- Rational Unified Process

46

46

Problem A - 需求.1

- 你是一個幫百視達(電影DVD租片商)進行電腦化的工程師。以下是你要處理的規格
- 百視達的片子分三種：新片(\$3)，舊片(\$2)，與兒童片(\$2)
- 一部電影有當然有很多片子可以承租
- 每一種片子有不同的計價。
 - 新片\$3 可以租一天，每多一天，一天是\$2
 - 舊片\$2 可以租三天，每多一天，一天是\$1
 - 兒童片\$2 可以租5天，每多一天，一天是\$1
- 每一種片子有不同的積分優惠來做為鼓勵常常光顧客戶的措施
 - 新片每租一片，常客基點 2.0，
 - 舊片每租一片，常客基點 1.0
 - 兒童片常客基點 0.5

47

47

Problem A - 需求.2

- 百視達常常會要求印出某一個客戶的報表 (print statement)，這個報表需要印出
 - 過去曾經租過的片子，所有租片的總消費金額 (請記得，一個客戶會租各種片子)
 - 計算常客積分總數

48

48

請分析之後寫出problem A

- Class 宣告
 - Class 名稱
 - 關鍵 attributes (member) 與常數
- 每個 Class 的關鍵 method (*)
 - Method 的名稱
 - Pseudo code
- 關鍵的 method print_statement()
 - 這個method 應該屬於哪一個 class
 - 其 pseudo code 應該如何用到其他的class 的關鍵 method (你在* 分析出來的)
- main()
 - 請展示如何使用你的class 來建立一些物件
 - 例如，如果一個客戶要租一隻片，要如何使用你的class 來建立物件與必要的關聯
 - 例如，如果客戶要求列印一個客戶報表，要如何呼叫你提供的服務

49

49

你要繳交的範例. 1

```
class A {
    string name;
    ...
    getPrice() {...}
};
class B {
    ...
    ...
    void print_statement() {
        for each movie a in B
            total += a.getPrice();
        print "過去租片總額="+total
    }
}
```

請注意

左邊只是個範例，你當然不能用沒有意義的名字來命名你的 class

print_statement 也只是個還差很遠的範例。詳細的 print_statement 應該完成的規格請看前幾頁

50

50

你要繳交的範例. 2

```
main() {
    ... // 請寫下如何用你的 classes 建立
        // 2 customers April, Joe
    ...// 請寫下如何用你的 classes 建立3 部電影
        // 分別是神力女超人(舊片)，正義聯盟(新片)，雷神索爾3(新片)
    ... // 請寫下如何用你的 classes 以及以上建立的
        // 物件實現以下的租片事實
        //
        //      1. April 租了神力女超人 3 天
        //      2. Joe 租了神力女超人 2 天
        //      3. April 租了正義聯盟 5 天
        //      4. Joe 租了雷神索爾3 1 天
    ... // 請寫下如何用你的 class
        // 印出 April 的客戶報表
        // 印出 Joe 的客戶報表
}
```

51

51